Proyecto 2a. Base de datos asíncrona con Python y MongoDB

Universidad Autónoma de Chihuahua Ingeniería en Ciencias de la Computación - 8CC2 Cómputo Paralelo y Distribuido Alumno: Adolfo Escobar Pereyra - **Matrícula:** 362839 **Profesor:** José Saúl de Lira Miramontes **Fecha:** noviembre de 2026

1. Objetivo

Implementar una aplicación de base de datos asíncrona en Python y MongoDB sobre `sample_mflix (movies)`, junto con su contraparte síncrona con las mismas funcionalidades, y comparar el desempeño de ambas.

2. Introducción y marco teórico

2.1 Programación síncrona vs asíncrona

En un programa **síncrono**, cada operación de red bloquea al hilo hasta recibir respuesta: si una consulta tarda 20 ms, el programa no hace nada durante esos 20 ms. En un programa **así ncrono** (`async`/`await`), cuando una operación debe esperar, el *event loop* aprovecha para avanzar otras tareas, así que N esperas se **traslapan** en lugar de sumarse.

Esto ayuda cuando el trabajo es **I/O-bound** (esperar red o disco), que es exactamente el caso de una base de datos. No ayuda (e incluso estorba) cuando el trabajo es **CPU-bound**, porque Python ejecuta un solo hilo a la vez por el GIL.

2.2 PyMongo y PyMongo Async

- **PyMongo clásico** (`MongoClient`): API bloqueante; cada llamada (`find_one`, `insert_many`, `bulk_write`) espera la respuesta.
- **PyMongo Async** (`AsyncMongoClient`, disponible desde PyMongo 4.10): la misma API pero con corrutinas; cada llamada se espera con `await` y el *event loop* puede atender otras operaciones mientras tanto. Requiere Python 3.9+ (este proyecto usa 3.11+ por `TaskGroup`).

2.3 Primitivas de asyncio utilizadas

- **`asyncio.gather(*coros)`**: ejecuta varias corrutinas en paralelo y devuelve sus resultados en orden. Ideal para lanzar un grupo de consultas independientes y esperarlas todas.
- **`asyncio.TaskGroup()`**: bloque `async with` donde se crean tareas

hijas con `create_task()`; al salir espera a todas y propaga errores de forma estructurada (si una falla, cancela el resto).

- **`asyncio.Semaphore(n)`**: limita a `n` las operaciones simultáneas.

Sin él, miles de tareas contra MongoDB saturarían el pool de conexiones y el servidor. Se usa con `async with sem:`.

- **`bulk_write([InsertOne|UpdateOne|DeleteOne...])`**: envía muchas

escrituras al servidor en una sola petición, mucho más eficiente que una por una. Se combina con lotes y concurrencia.

3. Diseño

3.1 Colecciones

Para **no dañar el dataset original**, el proyecto usa dos colecciones:

Colección	Uso
`sample_mflix.movies`	Solo **lectura** (consultas de demostración), si existe en el servidor
`proyecto2a.movies_carga`	**Escritura**: carga masiva, CRUD y benchmarks (seguro de borrar)

Si `sample_mflix` no existe (MongoDB vacío o Atlas sin sample dataset), las consultas usan automáticamente la colección de trabajo.

3.2 Esquema de documento

Subconjunto fiel de `sample_mflix.movies`: `title`, `year`, `runtime`, `rated`, `plot`, `fullplot`, `genres[]`, `directors[]`, `writers[]`, `cast[]`, `languages[]`, `countries[]`, `released`, `poster`, `imdb{rating, votes, id}`, `tomatoes.viewer{rating, numReviews}`, `num_mflix_comments`, `type`. Índices en `title`, `year`, `genres`, `imdb.rating` y `directors`.

3.3 Arquitectura

```
config.py                        URI, BD, clientes sync/async, índices
dataset/generador_peliculas.py   Generador determinista (semilla 362839)
sync_app/app_sync.py             SyncMoviesDB  (MongoClient)
async_app/app_async.py           AsyncMoviesDB (AsyncMongoClient)
comparacion/benchmark.py         Misma carga en ambas apps, mide tiempos
comparacion/demo_concurrencia.py Simulación de latencia I/O sin BD
scripts/verificar_conexion.py    Diagnóstico de conexión
scripts/cargar_sample_mflix.py   Importa un movies.json descargado
tests/test_sin_bd.py             15 pruebas con colecciones en memoria
```

Ambas clases exponen la **misma API** (mismos métodos y consultas) para que la comparación sea justa: lo único que cambia es el modelo de ejecución.

4. Implementación

4.1 Funcionalidades (idénticas en ambas apps)

- **Carga masiva**: partición en lotes (`en_lotes`) + `insert_many` o `bulk_write` con `InsertOne` por lote.

- **CRUD**: crear, leer por `_id`/título, listar paginado (`skip+limit` + orden), actualizar uno/muchos, eliminar uno/muchos, contar y vaciar la colección de trabajo.

- **Consultas**: por género, por rango de años, top-N por `imdb.rating` con mínimo de votos, búsqueda de título (regex insensible a mayúsculas), agregación por género (conteo + rating promedio), promedio por año y top de directores (pipelines `$match`/`$unwind`/ `$group`/`$sort`/`$limit`).

4.2 Dónde se usa cada primitiva exigida (app asíncrona)

Primitiva	Método	Qué hace
`TaskGroup`	`carga_masiva_taskgroup`	Una tarea hija por lote; cada una hace `bulk_write`
`TaskGroup`	`consultas_concurrentes_taskgroup`	N consultas como tareas hijas
`gather`	`carga_masiva_gather`	Variante de carga masiva con `gather`
`gather`	`consultas_concurrentes_gather`	Q consultas mezcladas en paralelo
`gather`	`bulk_actualizar_ratings`, `bulk_eliminar`	Lotes de `UpdateOne`/`DeleteOne` en paralelo
`Semaphore`	Todas las anteriores	Creado dentro de cada operación; tope de concurrencia
`bulk_write`	Carga, updates y deletes masivos	`InsertOne`/`UpdateOne`/`DeleteOne` por lote

Detalle de diseño: el `Semaphore` se crea **dentro de cada operación** y no en el constructor, porque queda ligado al *event loop* en uso y el objeto se reutiliza desde varios loops (el benchmark) o pruebas. El benchmark corre todo lo asíncrono en **un solo `asyncio.run`**.

4.3 Instalación y ejecución

```
cd Proyecto2a_MongoDB_362839
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env   # editar MONGO_URI si se usa Atlas
python -m scripts.verificar_conexion

# Demos automáticas
python -m sync_app.app_sync --demo --n 2000
```

```
python -m async_app.app_async --demo --n 2000 --concurrencia 10

# Benchmark comparativo (3 repeticiones, mejor tiempo)
python -m comparacion.benchmark --n 5000 --lote 1000 --consultas 50

# Pruebas sin BD + demo simulada
python -m unittest discover -s tests -v
python -m comparacion.demo_concurrencia --consultas 40 --concurrencia 10
```

MongoDB puede ser local (`mongod` o `docker run -d -p 27017:27017 mongo`) o Atlas (clúster gratuito; opcionalmente con *Load Sample Dataset*).

5. Resultados

5.1 Pruebas de lógica sin BD: 15/15 correctas

`tests/test_sin_bd.py` verifica con colecciones simuladas en memoria: esquema y determinismo del generador, partición en lotes, CRUD completo, `bulk_write` de inserción/actualización/borrado, las 7 consultas y agregaciones, paginación, carga masiva con `TaskGroup` y con `gather`, consultas concurrentes, y que el `Semaphore` limita la concurrencia real (máximo observado <= 4 con tope 4, y > 1, es decir, sí hubo paralelismo). Además se verificó que el benchmark aplica **la misma carga de trabajo** en ambas ramas (10 consultas -> 90 docs en ambas; 100 updates en ambas), y un test de paridad garantiza que los fakes replican la API real (`aggregate()` es corutina, `find()` no), para no enmascarar bugs.

5.2 Demo simulada de concurrencia (medida, sin BD)

Con latencia I/O simulada de ~50 ms por consulta, 40 consultas, concurrencia 10 (corre con `python -m comparacion.demo_concurrencia`):

Versión	Tiempo	Speedup
Síncrona secuencial	1.66 s	x1.00
`gather` + `Semaphore`	0.19 s	x8.92
`TaskGroup` + `Semaphore`	0.19 s	x8.95

Con trabajo I/O-bound, traslapar esperas reduce el tiempo total en proporción cercana a la concurrencia configurada.

5.3 Benchmark real contra MongoDB Atlas (medido)

Comando: `python -m comparacion.benchmark --n 5000 --consultas 50` (MongoDB Atlas 8.0, concurrencia 10, lotes de 1000, mejor de 3 repeticiones por experimento; evidencia completa en `comparacion/resultados_benchmark.json`).

Experimento	Síncrono (s)	Asíncrono (s)	Speedup	Ganador

A. Carga masiva (N=5000)	13.45	6.47	x2.08	ASYNC
B. Consultas mezcladas (Q=50)	9.83	1.14	x8.65	ASYNC
C. Update masivo (2000)	5.55	1.13	x4.90	ASYNC

Lectura: la app asíncrona gana en los tres experimentos. El speedup es máximo en B (x8.65, cercano a la concurrencia 10: 50 consultas independientes donde domina la latencia de red) y notable en C (x4.90). En A (x2.08) el traslape de lotes también ayuda, aunque cada `bulk_write` ya es eficiente por sí mismo. Contra Atlas (decenas de ms de latencia) el beneficio es mayor que contra un MongoDB local (~1 ms), donde el límite pasa a ser el servidor.

6. Comparación síncrona vs asíncrona

Aspecto	**Síncrona (PyMongo)**	**Asíncrona (PyMongo Async)**
Modelo	Una operación a la vez; bloqueo por llamada	Muchas operaciones traslapadas en un hilo
Código	Simple, directo	Requiere `async`/`await` y diseño de concurrencia
Carga masiva	Lotes secuenciales	Lotes concurrentes (`TaskGroup` + `Semaphore`)
Múltiples consultas	Suma de latencias	Latencia traslapada (`gather`)
Throughput I/O-bound	Bajo	Alto (limitado por `Semaphore` y el servidor)
Riesgos	Ninguno particular	Exceso de concurrencia sin `Semaphore`; depuración más compleja
Cuándo usarla	Scripts simples, pocas operaciones	Servidores, cargas masivas, muchas consultas

7. Conclusiones

- Se implementaron las dos aplicaciones con **funcionalidades idénticas** (carga masiva, CRUD y 7 consultas/agregaciones) sobre el esquema de `sample_mflix.movies`, usando `MongoClient` y `AsyncMongoClient`.

- La app asíncrona emplea las cuatro técnicas pedidas: `gather`, `TaskGroup`, `Semaphore` y `bulk_write`, localizables en la tabla de la sección 4.2.

- La comparación es justa (misma API, misma carga, mismo estado inicial por experimento) y reproducible con un solo comando; la medición simulada confirma speedups cercanos a la concurrencia en trabajo I/O-bound.

- Como buena práctica, las escrituras van a una colección de trabajo separada y el `sample_mflix` original solo se lee.

8. Referencias

- Documentación de PyMongo (MongoClient y AsyncMongoClient).
- Documentación de Python: asyncio (gather, TaskGroup, Semaphore).
- Documentación de MongoDB: operadores de agregación y bulkWrite.